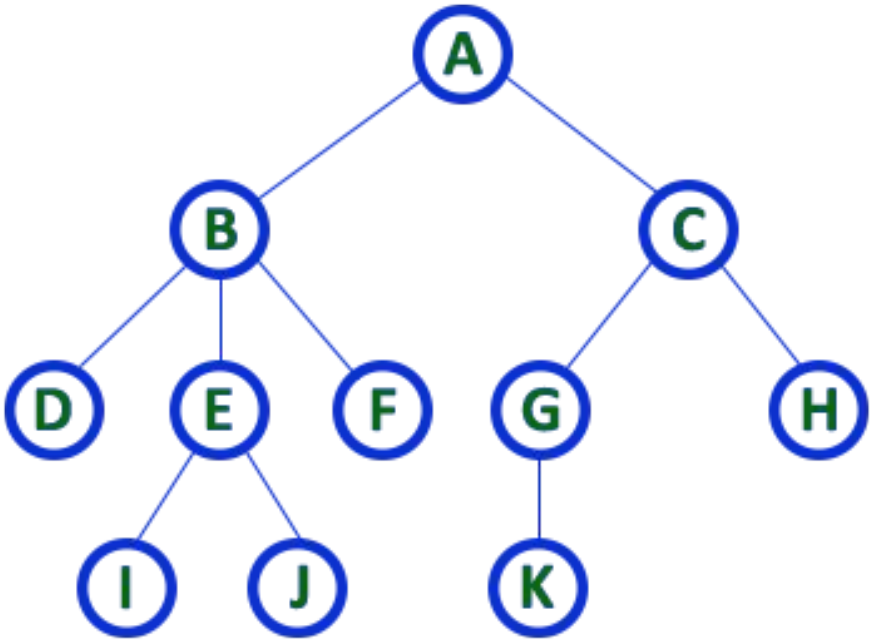
Trees

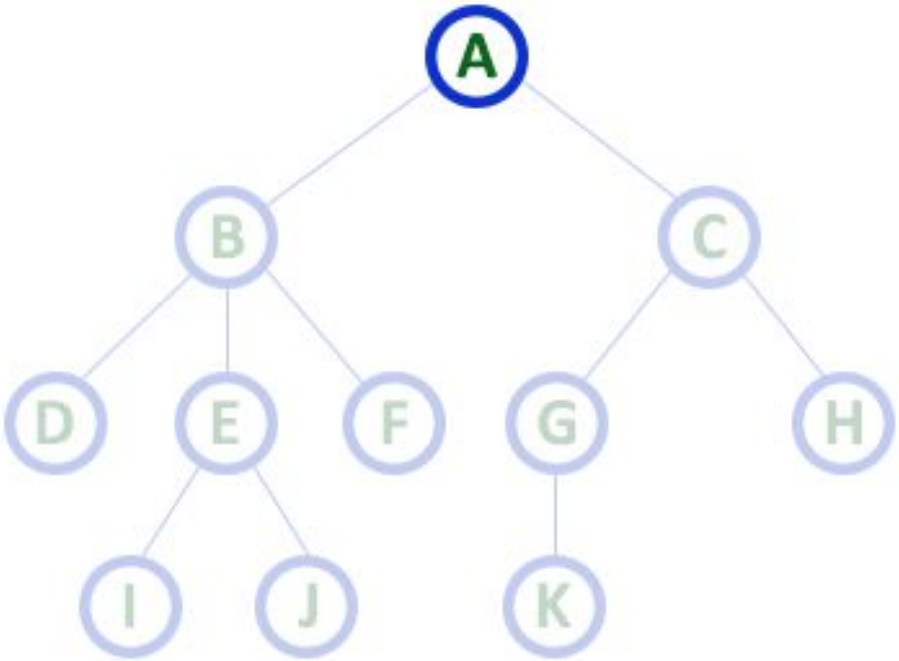


TREE with 11 nodes and 10 edges

- In any tree with 'N' nodes there will be maximum of 'N-1' edges
- In a tree every individual element is called as 'NODE'

Root

In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. We can say that the root node is the origin of the tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree

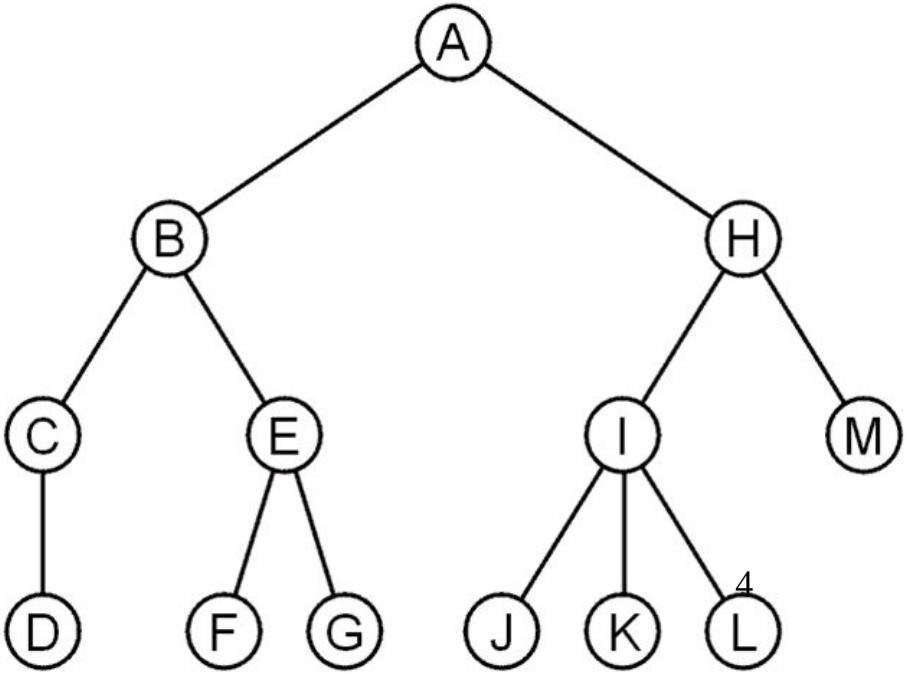


Here 'A' is the 'root' node

- In any tree the first node is called as ROOT node

A rooted tree data structure stores information in *nodes*

- Similar to linked lists:
  - There is a first node, or *root*
  - Each node has variable number of references to successors
  - Each node, other than the root, has exactly one node pointing to it

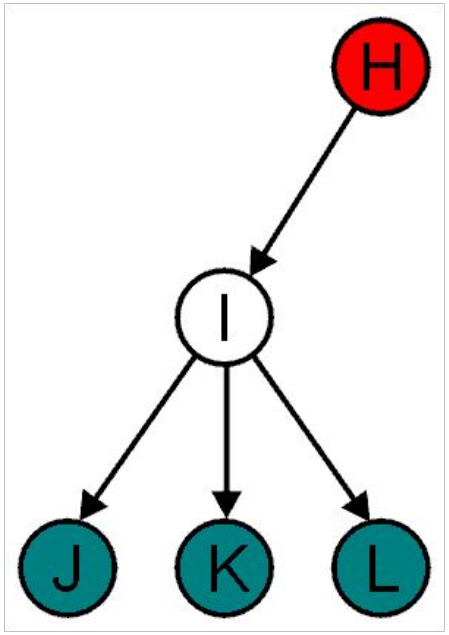


All nodes will have zero or more child nodes or *children*

- I has three children: J, K and L

For all nodes other than the root node, there is one parent node

- H is the parent I

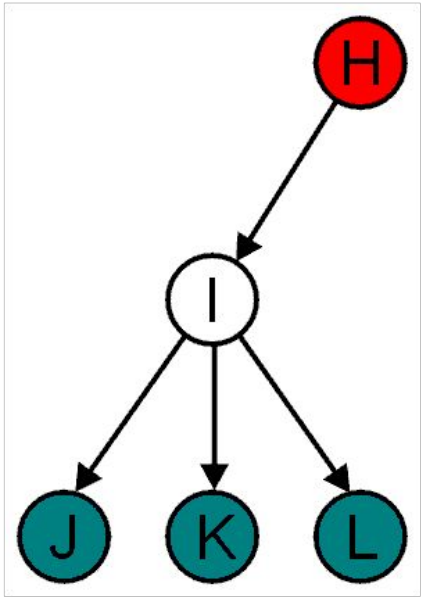


The *degree* of a node is defined as the number of its children:

$$\deg(I) = 3$$

Nodes with the same parent are *siblings*

– J, K, and L are siblings



Leaf

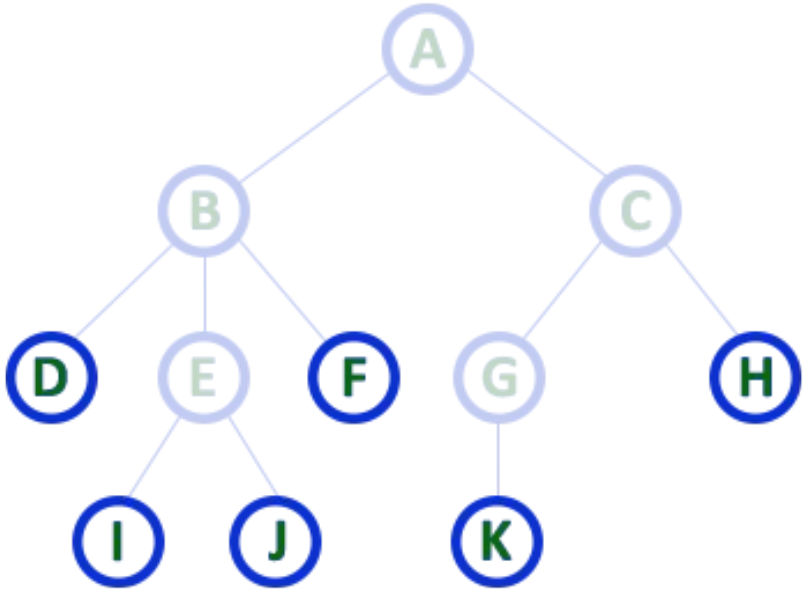
In a tree data structure, the node which does not have a child is called as **LEAF Node**. In simple words, a leaf is a node with no child.

In a tree data structure, the leaf nodes are also called as **External Nodes**. External node is also a node with no child. In a tree, leaf node is also called as '**Terminal**' node.

Internal Nodes

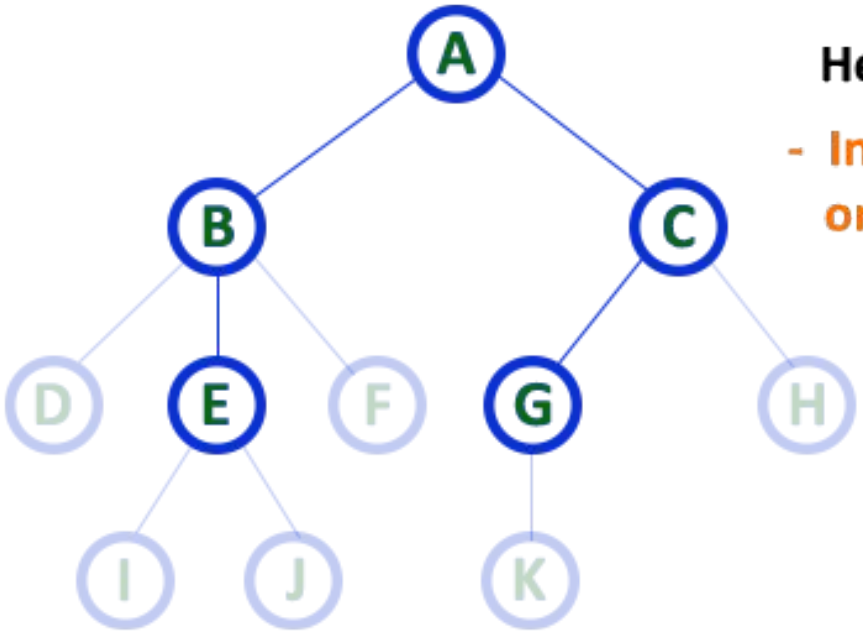
In a tree data structure, the node which has atleast one child is called as **INTERNAL Node**. In simple words, an internal node is a node with atleast one child.

In a tree data structure, nodes other than leaf nodes are called as **Internal Nodes**. **The root node is also said to be Internal Node** if the tree has more than one node. Internal nodes are also called as '**Non-Terminal**' nodes.



Here D, I, J, F, K & H are Leaf nodes

- In any tree the node which does not have children is called 'Leaf'
- A node without successors is called a 'leaf' node



Here A, B, C, E & G are Internal nodes

- In any tree the node which has atleast one child is called 'Internal' node
- Every non-leaf node is called as 'Internal' node

Edge

In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.

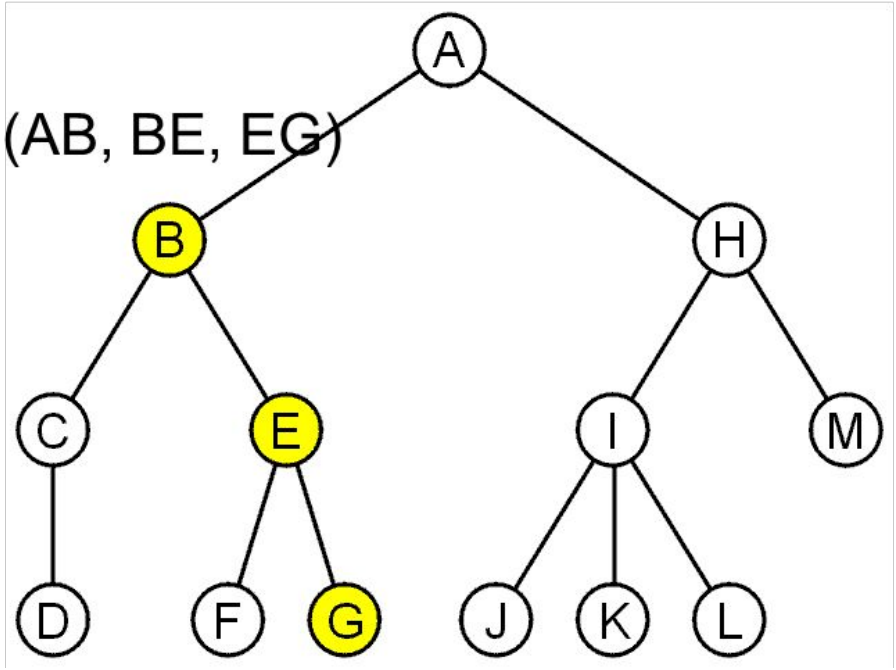
A path is a sequence of nodes

$$(a_0, a_1, \ldots, a_n)$$

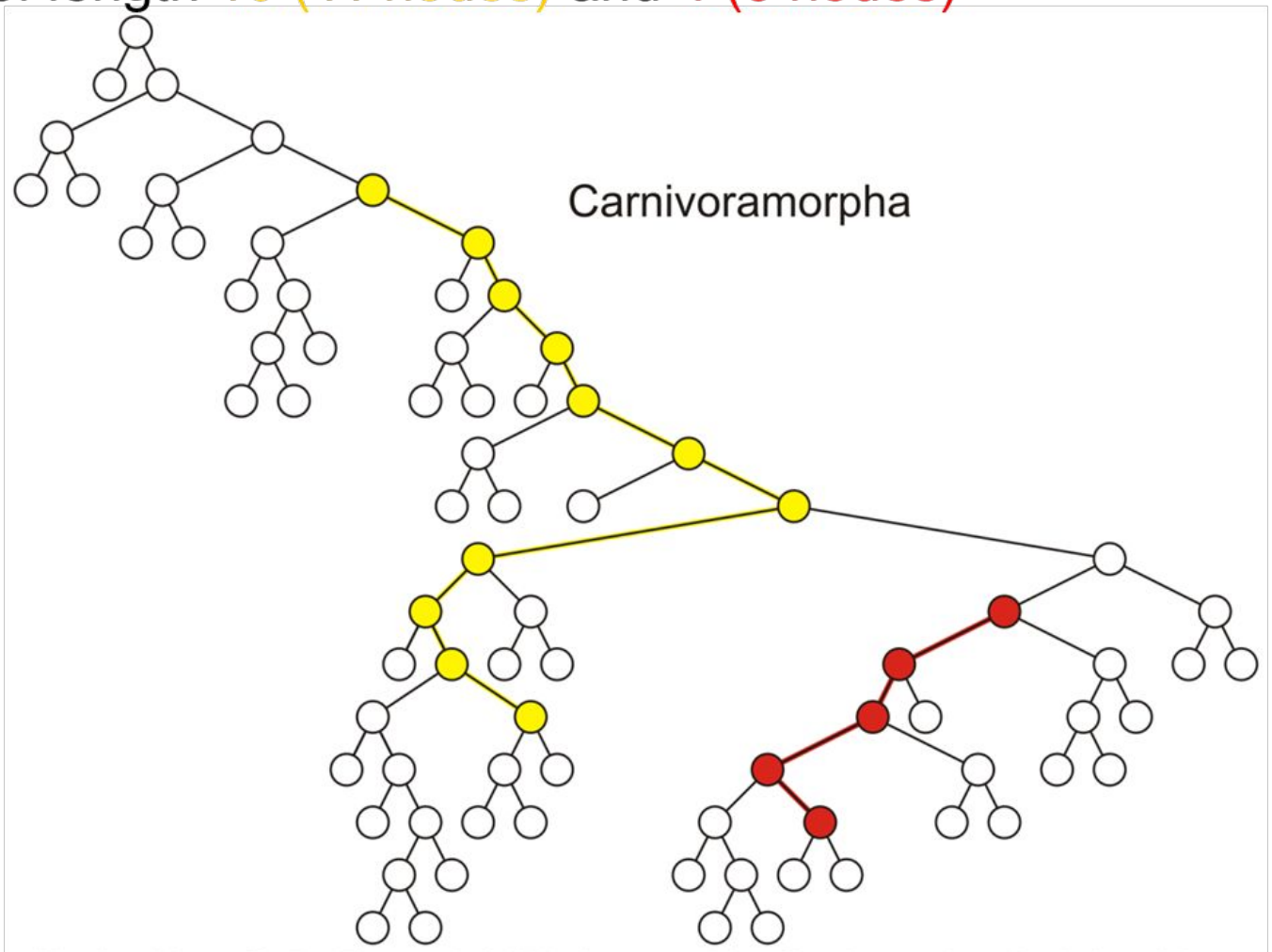
where a_{k+1} is a child of a_k is

The length of this path is n

E.g., the path (AB, BE, EG) has length 2



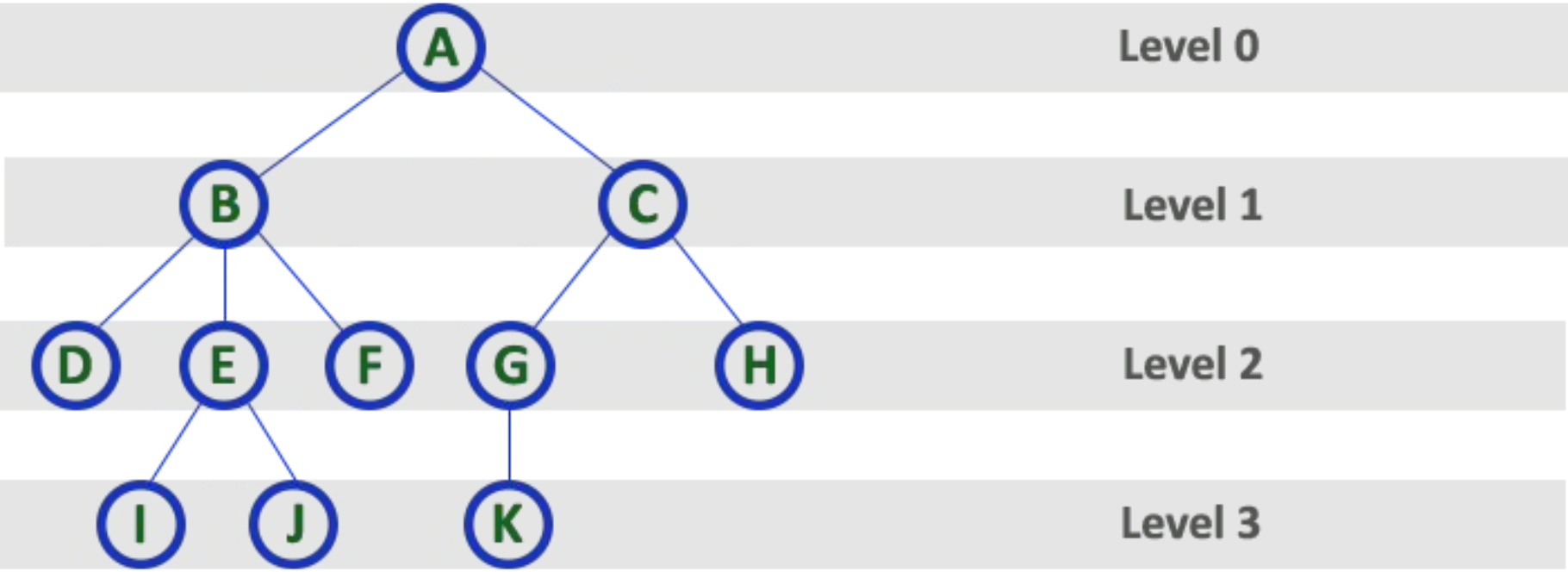
Paths of length 10 (11 nodes) and 4 (5 nodes)



Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorphans, and assessment of the position of 'Miacoidea'

Level

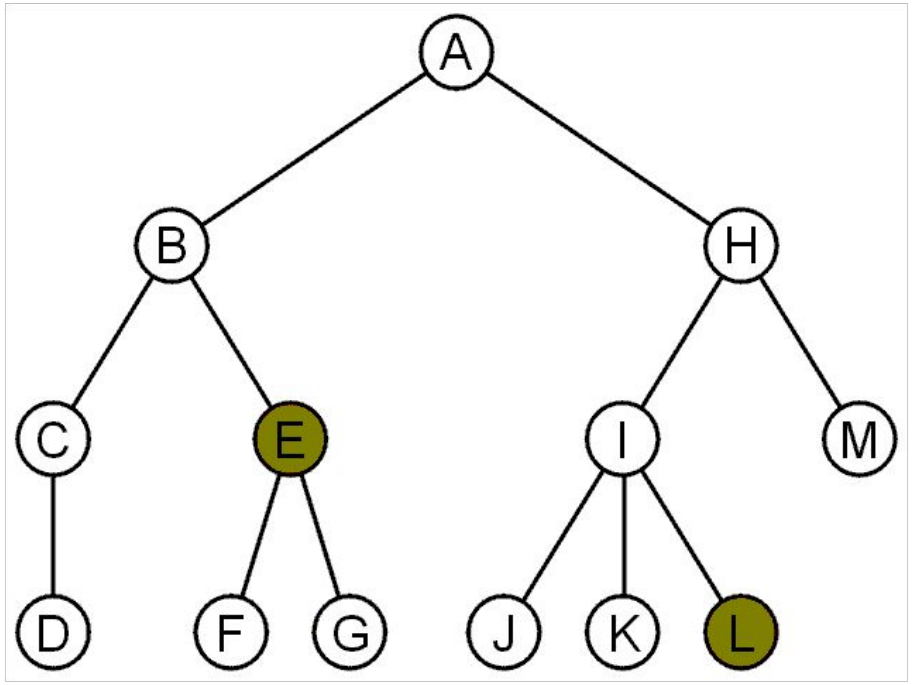
In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



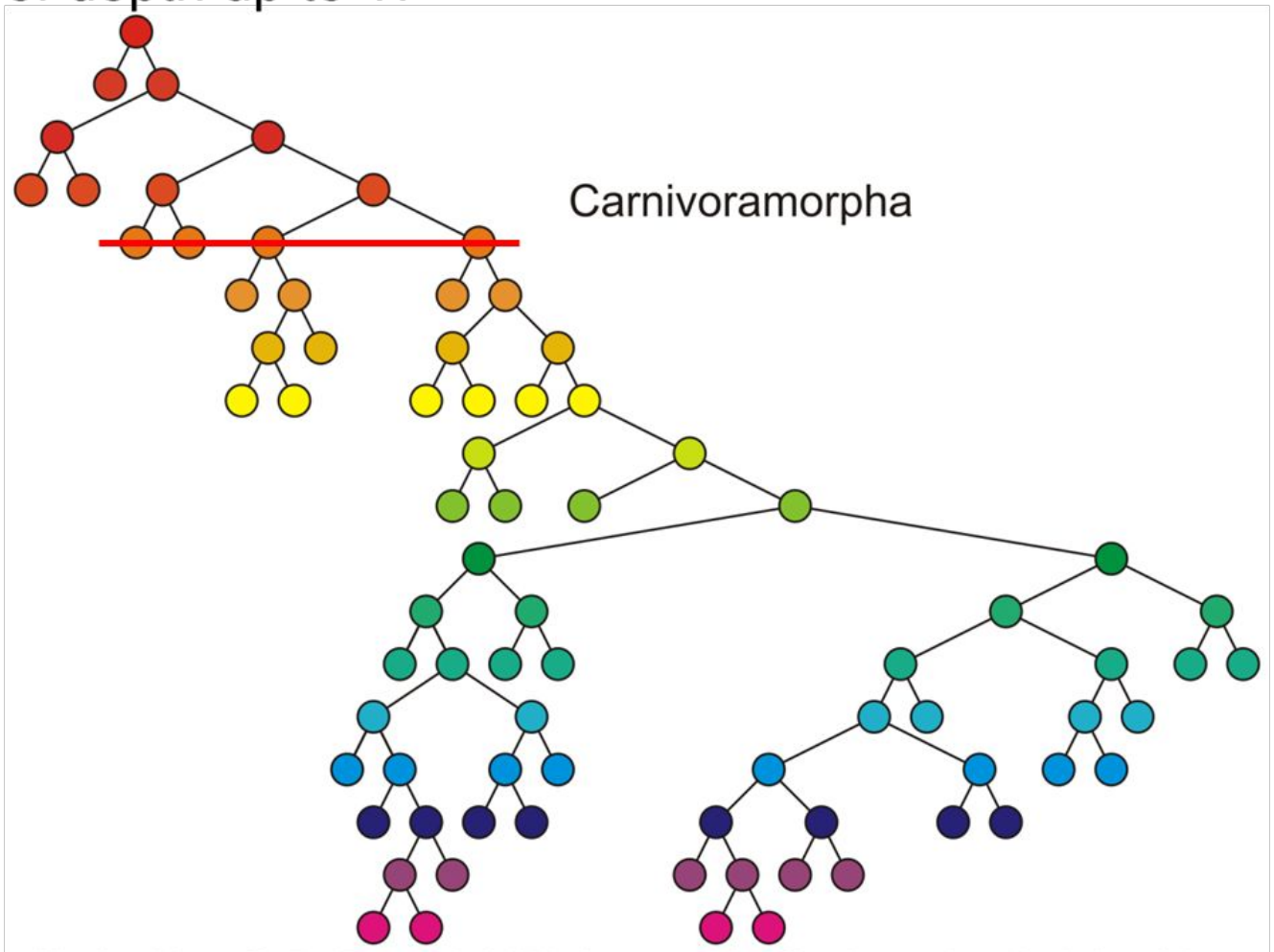
For each node in a tree, there exists a unique path from the root node to that node

The length of this path is the *depth* of the node, *e.g.*,

- E has depth 2
- L has depth 3



Nodes of depth up to 17



Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorphans, and assessment of the position of 'Miacoidea'

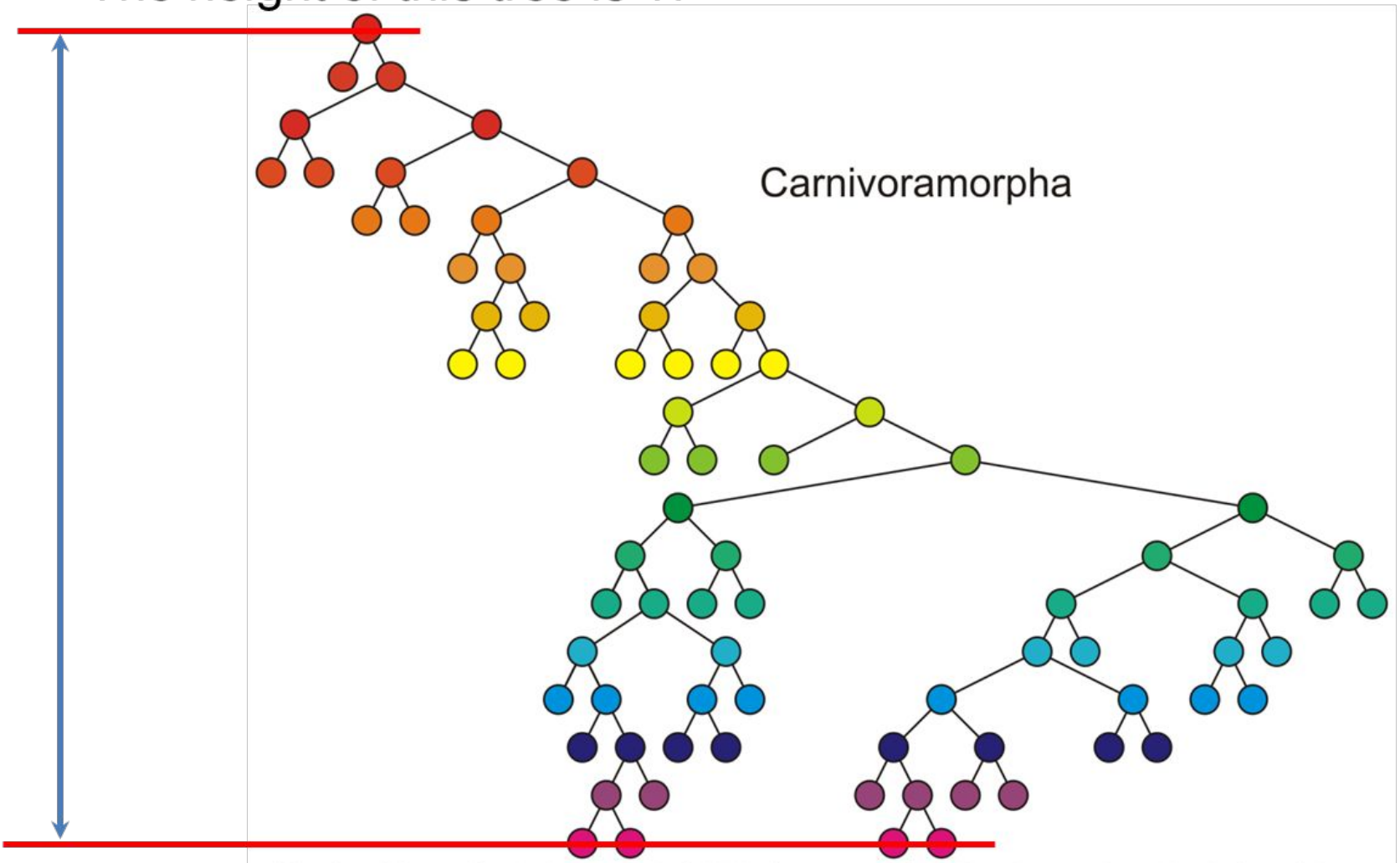
The ***height*** of a tree is defined as the maximum depth of any node within the tree

The height of a tree with one node is 0

– Just the root node

For convenience, we define the height of the empty tree to be –1

The height of this tree is 17



Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorphans, and assessment of the position of 'Miacoidea'

If a path exists from node *a* to node *b*:

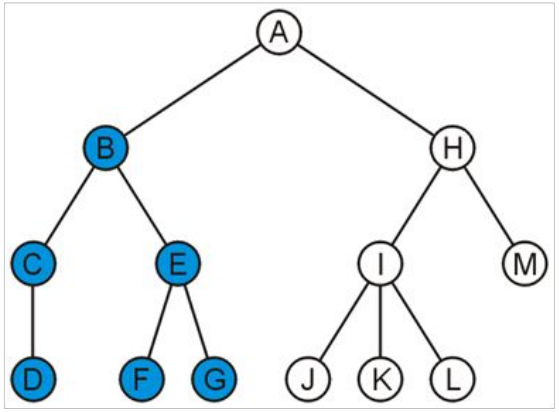
- *a* is an ***ancestor*** of *b*
- *b* is a ***descendent*** of *a*

Thus, a node is both an ancestor and a descendant of itself

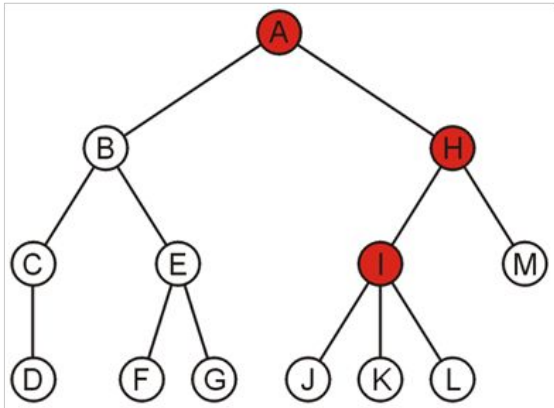
- We can add the adjective *strict* to exclude equality: *a* is a *strict descendent* of *b* if *a* is a descendant of *b* but $a \neq b$

The root node is an ancestor of all nodes

The descendants of node B are B, C, D, E, F, and G:

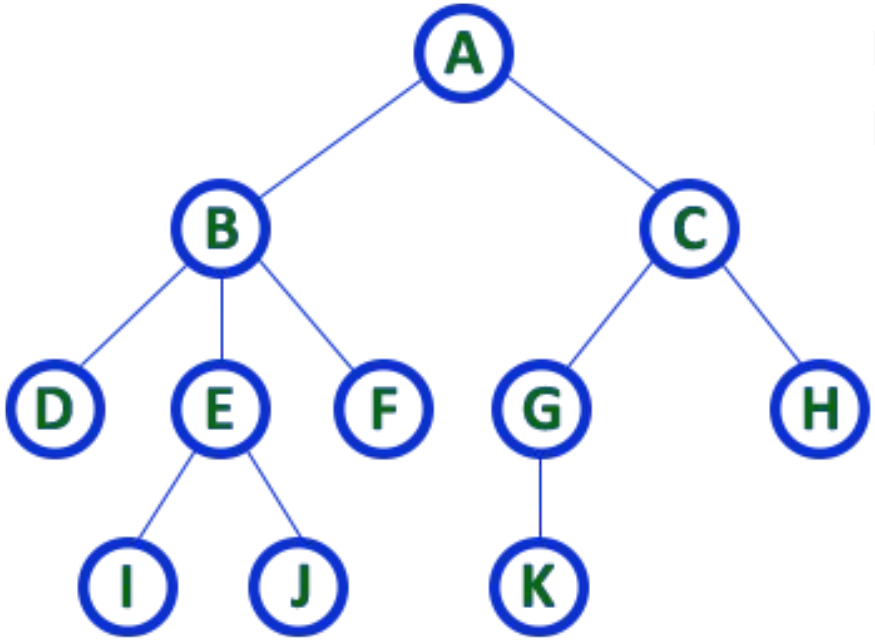


The ancestors of node I are I, H, and A:



Degree

In a tree data structure, the total number of children of a node is called as DEGREE of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'



Here Degree of B is 3

Here Degree of A is 2

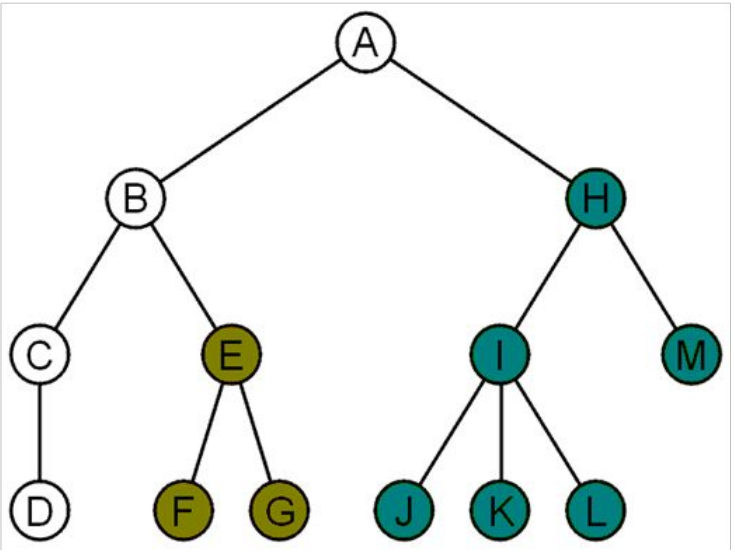
Here Degree of F is 0

- In any tree, 'Degree' of a node is total number of children it has.

Another approach to a tree is to define the tree recursively:

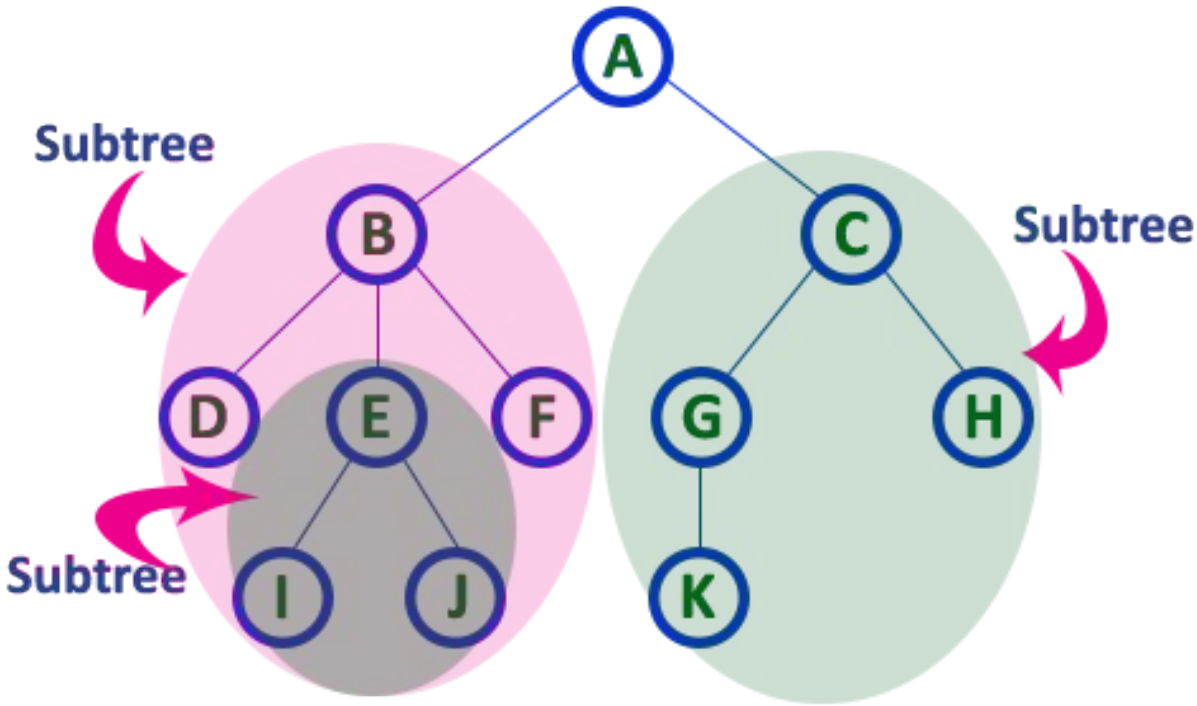
- A degree-0 node is a tree
- A node with degree n is a tree if it has n children and all of its children are disjoint trees (*i.e.*, with no intersecting nodes)

Given any node a within a tree with root r, the collection of a and all of its descendants is said to be a *subtree of the tree with root* a



Sub Tree

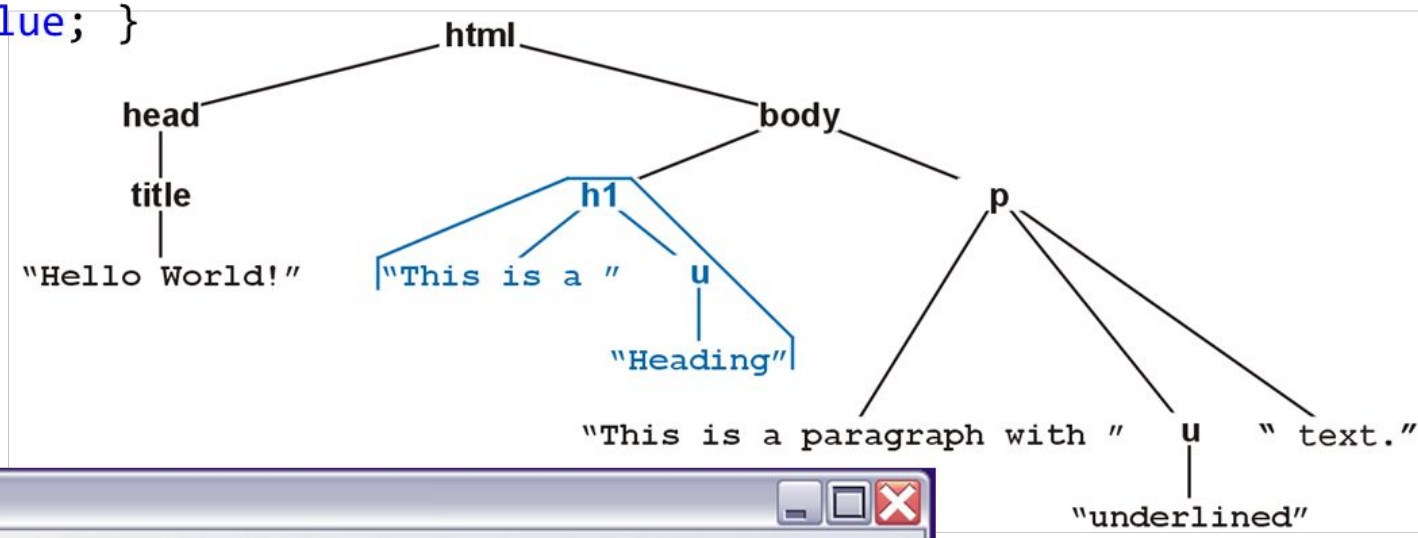
In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



Example: XHTML and CSS

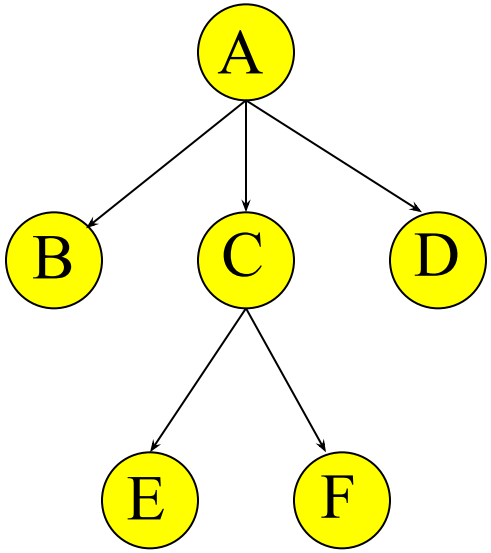
For example, this style renders as follows:

```
<style type="text/css">
   h1 { color:blue; }
</style>
```



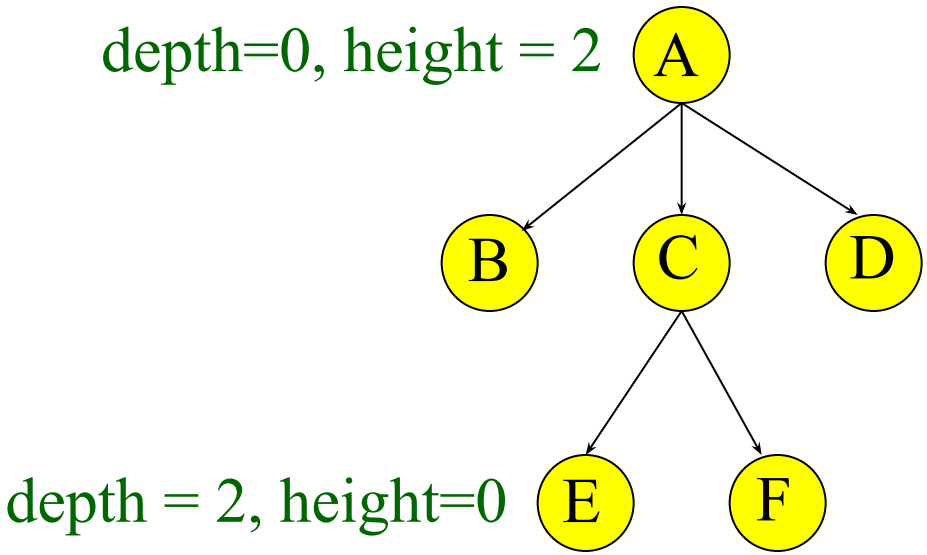
Tree terminology cont--

- Nodes: A, B, …, F
- Root node: A
- Leaf nodes: B, E, F, D
- Edges: (A,B), (A,C), …, (C, F)
- Path: sequence of nodes connected by edges.
- Path examples: (AB),(AC),(AD), (AC,CE), (AC,CF), (CE,CF)



Questions. A tree has N nodes.
How many edges does it have ? How many paths ?

- **Length** of a path = number of edges
- **Depth** of a node x = length of path from root to x
- **Height** of node x = length of longest path from x to a leaf
- Depth and height of tree = height of root
- The label of a node: A, B, C, …



Definition

A tree is a collection of elements called ***nodes***, one of which is distinguished as a ***root***, along with a relation ("parenthood") that places a hierarchical structure on the nodes.

A node, like an element of a list, can be of whatever type we wish. We often depict a node as a letter, a string, or a number with a circle around it. Formally, a ***tree*** can be defined recursively in the following manner.

1. A single node by itself is a tree. This node is also the root of the tree.
2. Suppose ***n*** is a node and $T1, T2, \ldots, Tk$ are trees with roots $n1, n2, \ldots, nk$, respectively. We can construct a new tree by making n be the parent of nodes $n1, n2, \ldots, nk$. In this tree ***n*** is the root and $T1, T2, \ldots, Tk$ are the ***subtrees*** of the root. Nodes $n1, n2, \ldots, nk$ are called the *children* of node n.

Definition

If there is a path from node *a* to node *b*, then *a* is an **ancestor** of *b*, and *b* is a **descendant** of *a*.

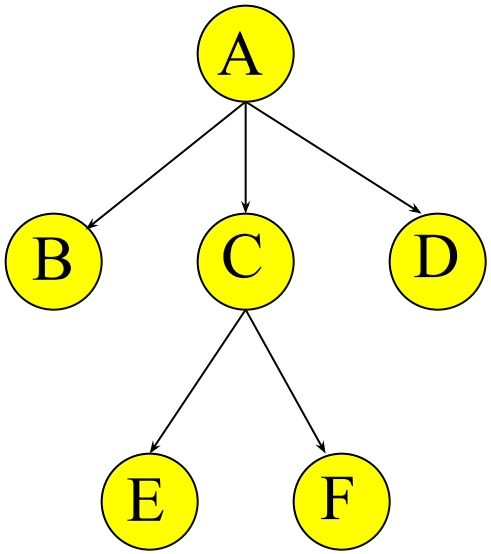
An ancestor or descendant of a node, other than the node itself, is called a **proper** ancestor or proper descendant, respectively.

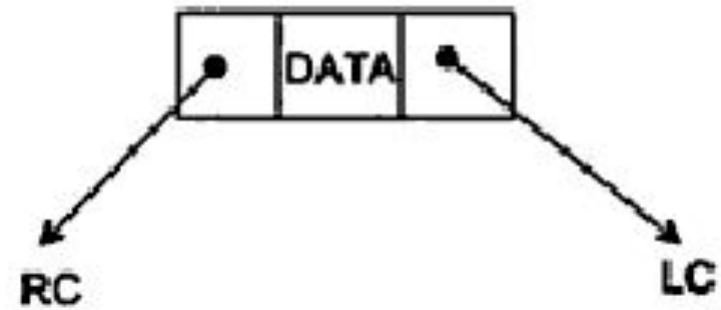
In a tree, the root is the only node with no proper ancestors. A node with no proper descendants is called a **leaf**.

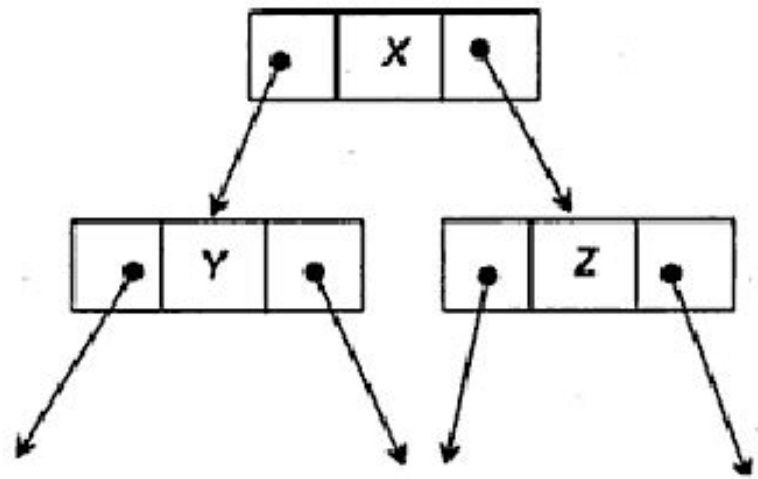
A **subtree** of a tree is a node, together with all its descendants.

The **height** of a node in a tree is the length of a longest path from the node to a leaf.

Implementation of Trees



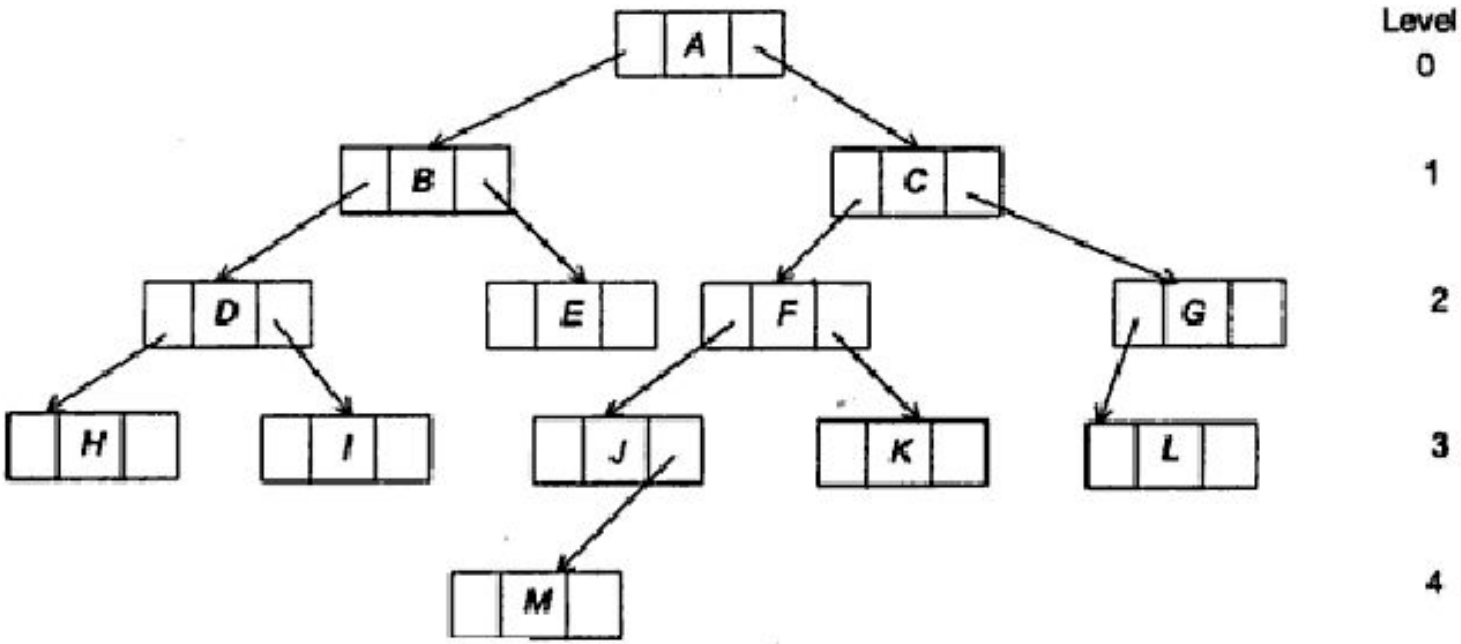




Binary trees

A binary tree is an important type of tree structure which occurs very often. It is characterized by the fact that any node can have at most two branches, i.e., there is no node with degree greater than two.

For binary trees we distinguish between the subtree on the left and on the right, whereas for trees the order of the sub trees was irrelevant. Also a binary tree may have zero nodes.



A binary tree may also be defined as follows:

- A binary tree is either an empty tree
- Or a binary tree consists of a node called the root node, a left subtree and a right subtree, both of which will act as a binary tree once again.

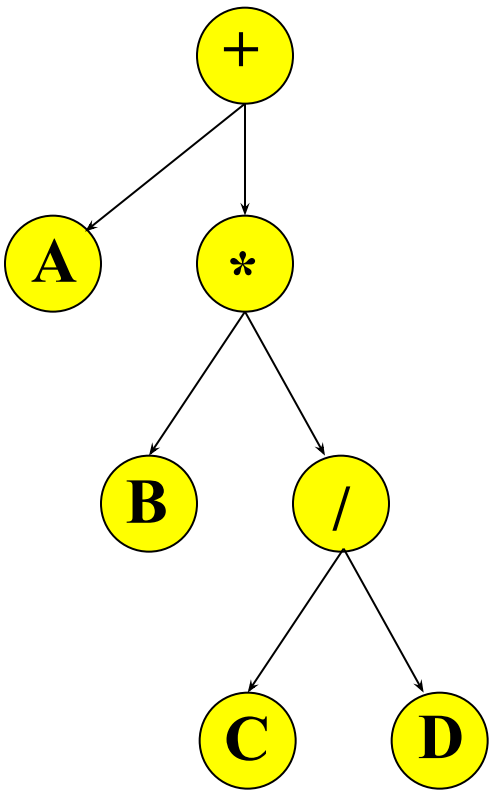
- Applications : Binary trees are used to represent a nonlinear data structure. There are various forms of Binary trees. Binary trees play a vital role in a software application. One of the most important applications of the Binary tree is in the searching algorithm

Application: Arithmetic Expression Trees

Example Arithmetic Expression:

A + (B * (C / D))

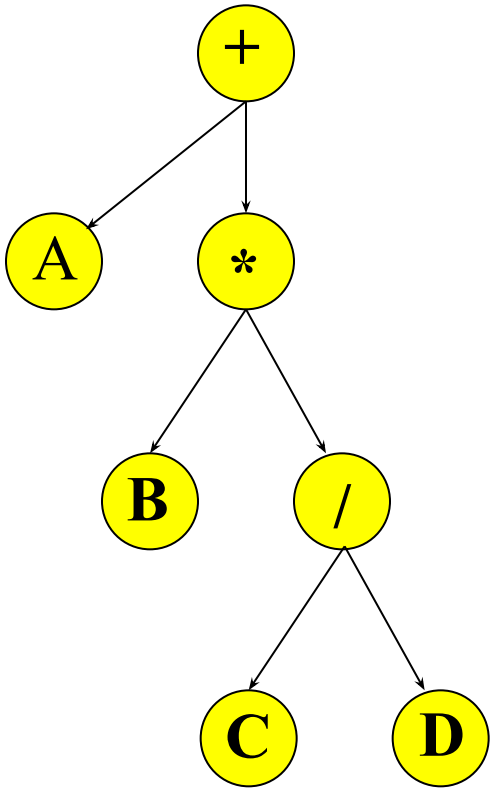
Tree for the above expression:



- Used in most compilers
- No parenthesis need – use tree structure
- Can speed up calculations e.g. replace / node with C/D if C and D are known
- Calculate by traversing tree (how?)

Traversing Trees

- **<u>Preorder</u>: Root, then Children**
  - + A * B / C D
- **<u>Postorder</u>: Children, then Root**
  - A B C D / * +
- **Inorder: Left child, Root, Right child**
  - A + B * C / D



Example Code for Recursive Preorder

```
void print_preorder ( TreeNode T)
{ if  ( T == NULL ) return;
  print_element(T.Element());
  print_preorder(T.FirstChild());
  print_preorder(T.NextSibling());
}
```

What is the running time for a tree with N nodes?

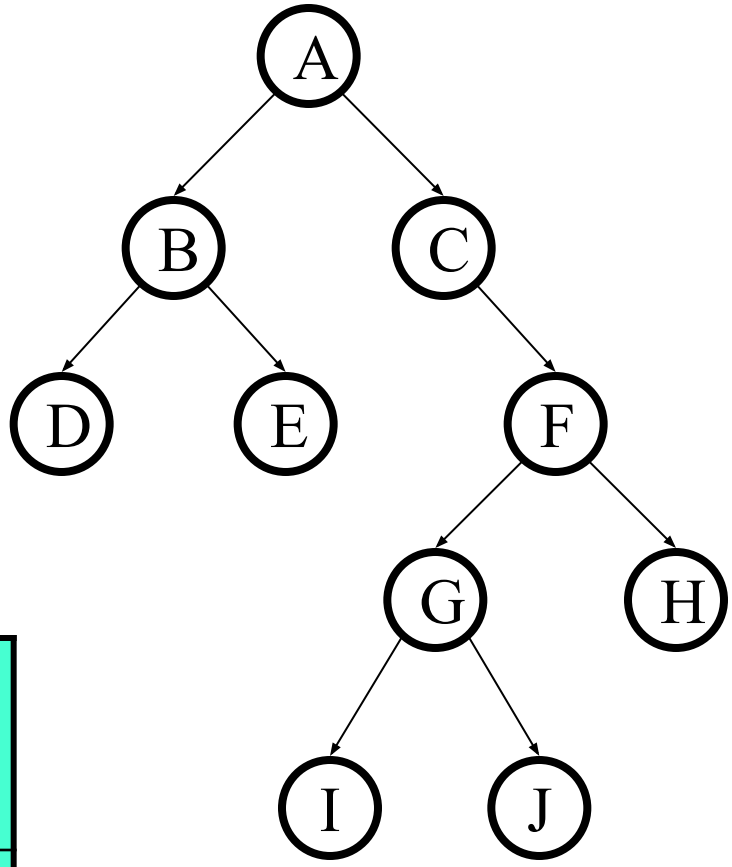
- PreOrder traversal - visit the parent first and then left and right children;

- InOrder traversal - visit the left child, then the parent and the right child;

- PostOrder traversal - visit left child, then the right child and then the parent;

Binary Trees

- **Properties**
  - max # of leaves = $2^{depth(tree)}$
  - max # of nodes = $2^{depth(tree)+1} - 1$
- **We care a lot about the depth:**
  - max depth = n-1
  - min depth = log(n)
  - average depth for n nodes = $\sqrt{n}$ (over all possible binary trees)
- **Representation:**

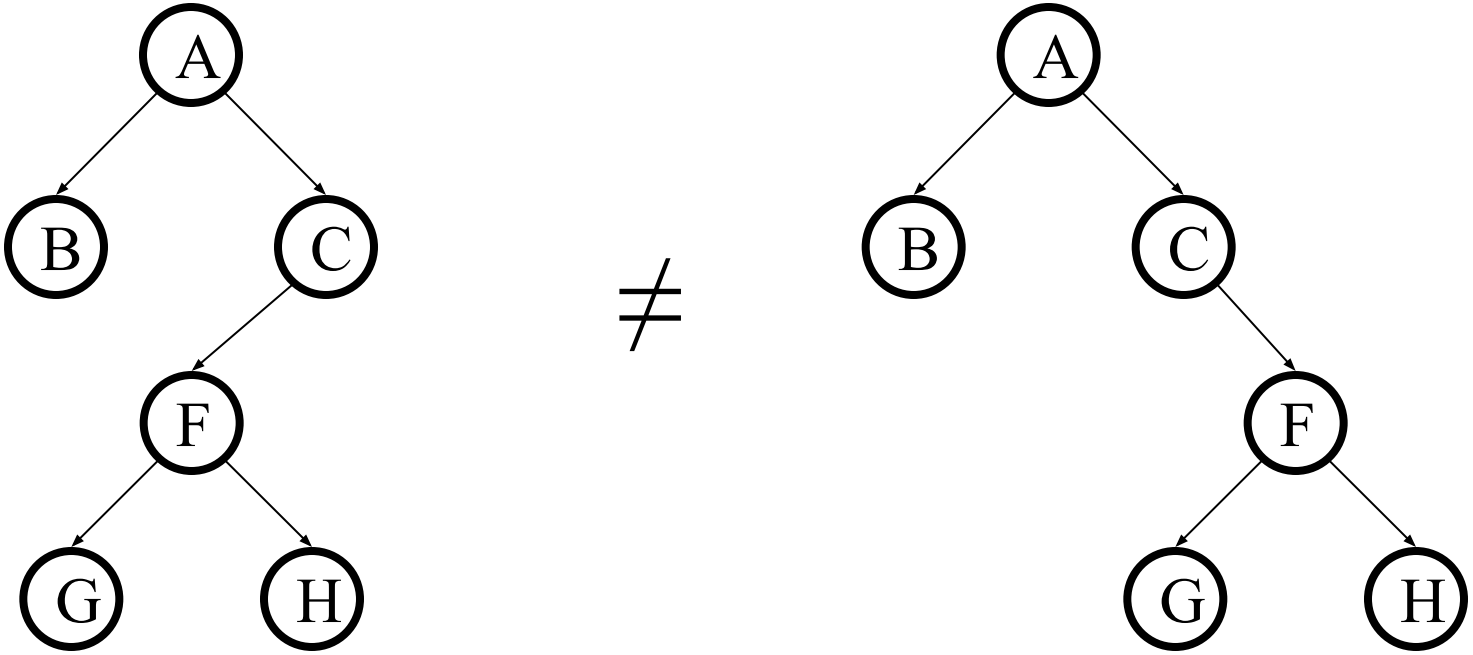


TreeNode:	Element	
	Left	Right

Binary Trees

Notice:

- we distinguish between *left* child and *right* child



Types of Binary Trees

A <u>full binary tree</u> which is also called as proper binary tree or 2-tree is a tree in which all the node other than the leaves has exact two children.

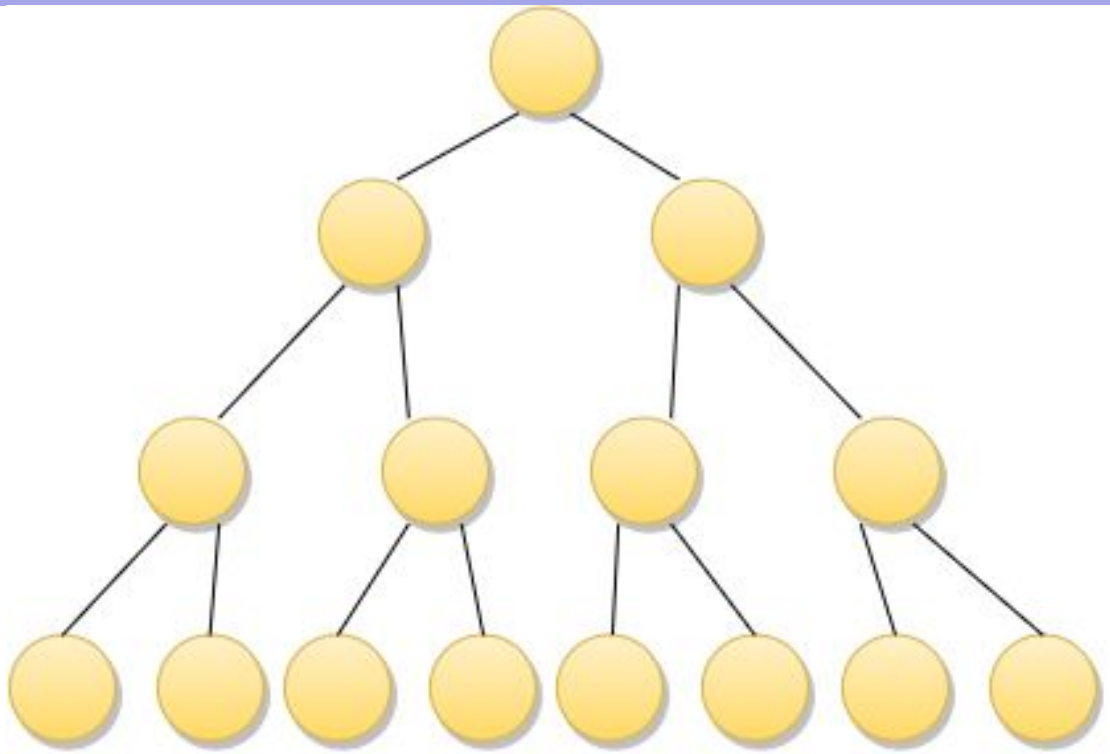


Fig: Full Binary Tree

w3schools.in

A **complete binary tree** is a binary tree in which at every level, except possibly the last, has to be filled and all nodes are as far left as possible.

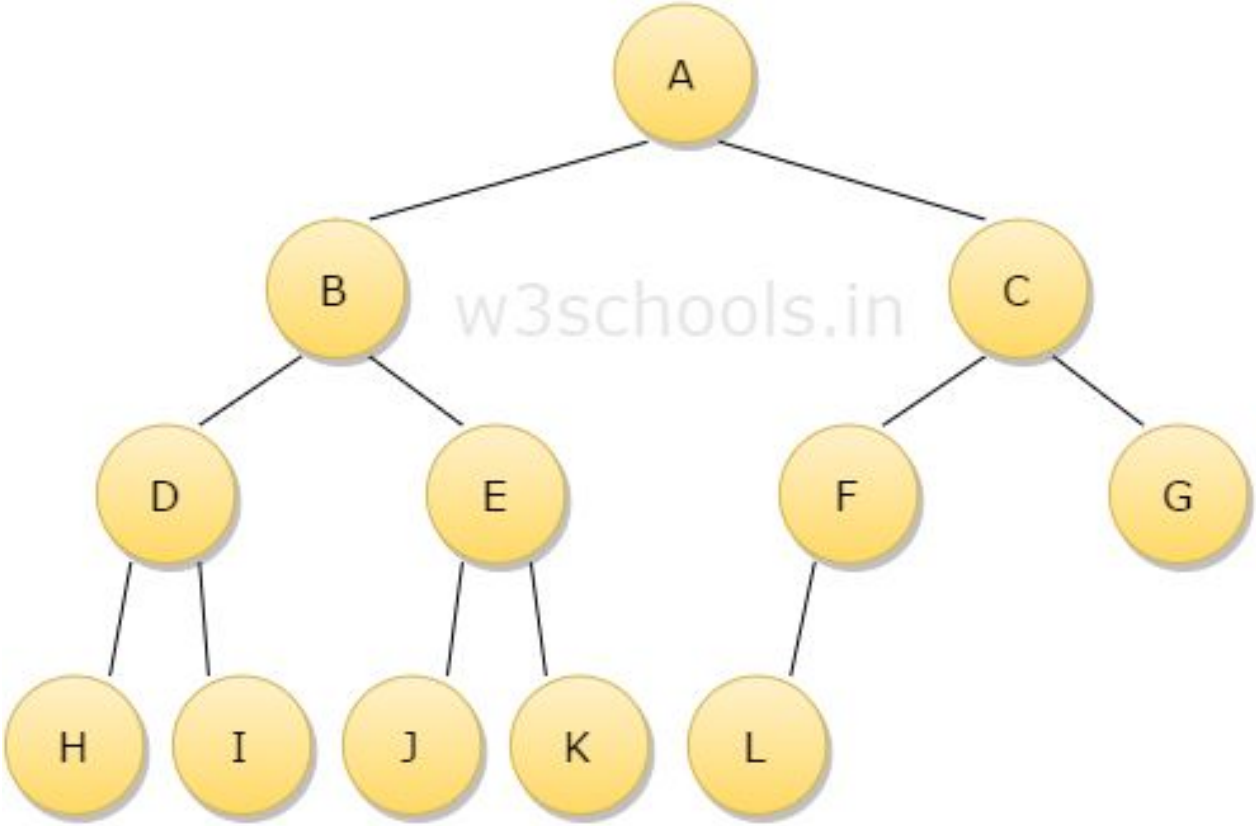


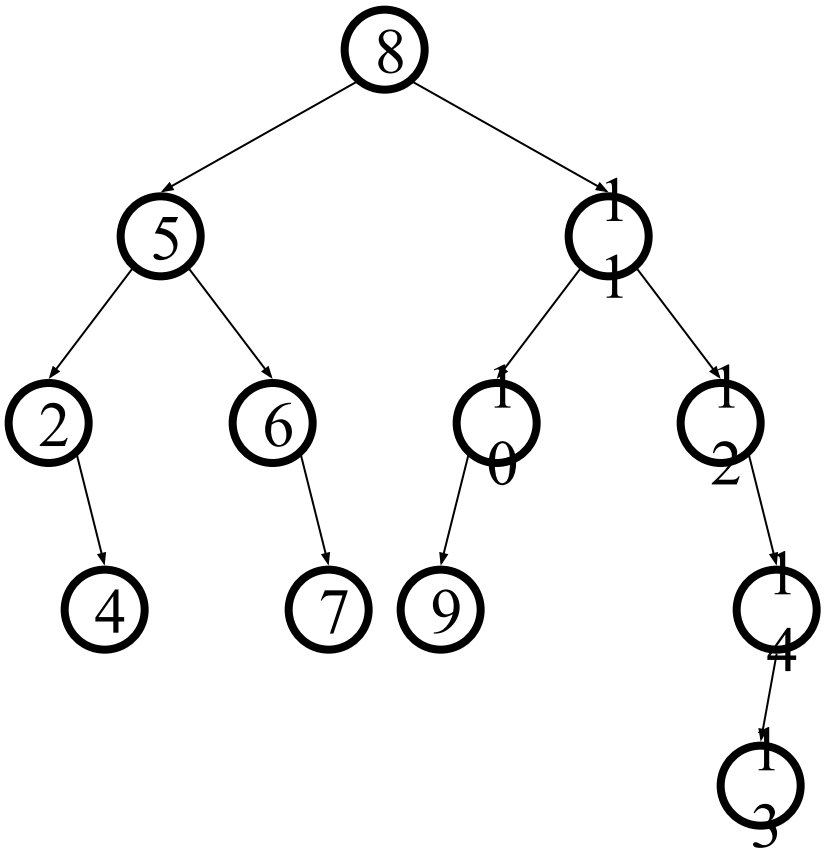
Fig: Complete Binary Tree

A binary tree can be converted into an **extended binary** tree by adding new nodes to its leaf nodes and to the nodes that have only one child. These new nodes are added in such a way that all the nodes in the resultant tree have either zero or two children. It is also called **2 - tree**.

The **threaded Binary tree** is the tree which is represented using pointers the empty subtrees are set to NULL, i.e. 'left' pointer of the node whose left child is empty subtree is normally set to NULL. These large numbers of pointer sets are used in different ways.

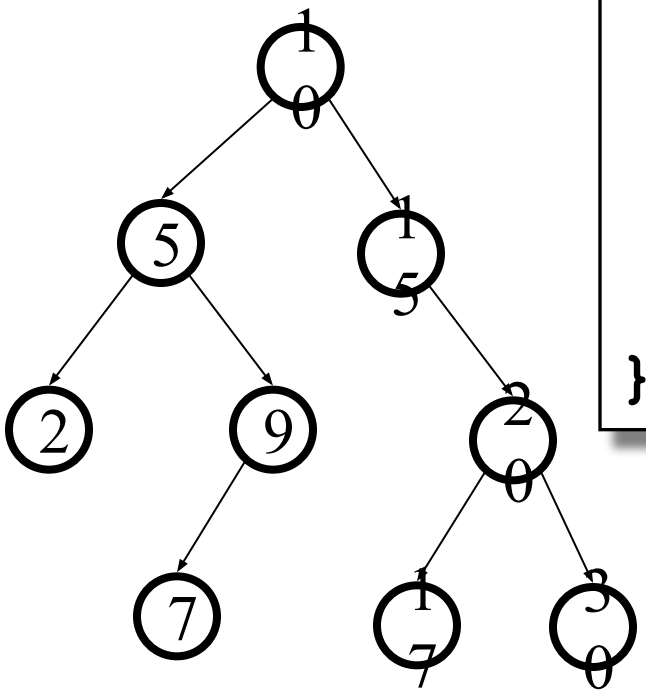
Binary Search Tree

- Search tree property
  - **all keys in left subtree smaller than root's key**
  - **all keys in right subtree larger than root's key**



  - result:
    - easy to find any given key
    - inserts/deletes by changing links

Searching in a Binary Search Tree

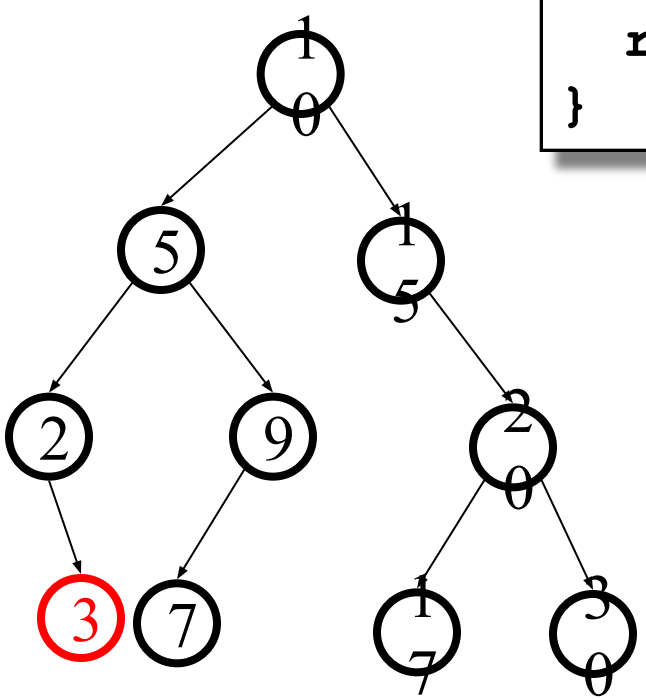



```
Boolean find(int x, TreeNode
  T)
{ if  ( T == NULL )
      return false;
  if (x == T.Element)
      return true;
  if (x < T.Element)
      return find(x, T.Left);
  return find(x, T.Right);
}
```

What is the running time ?

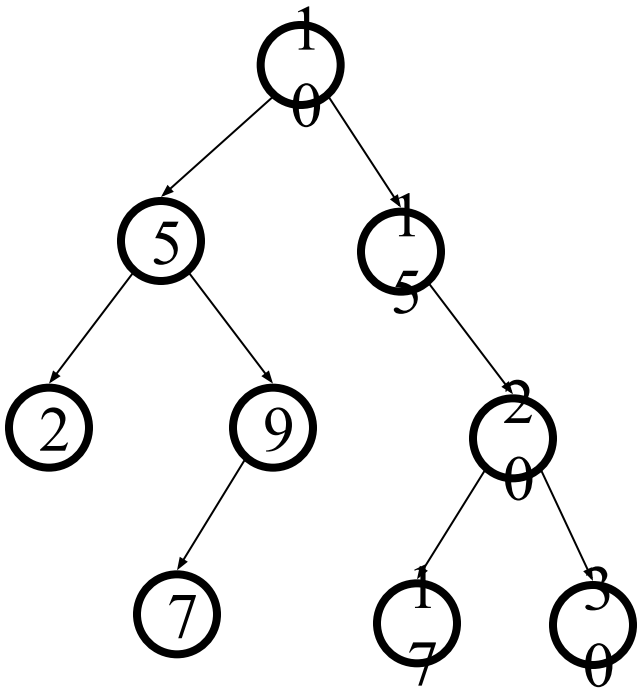
Insert a Key

```
TreeNode insert(int x, TreeNode T)
{ if  ( T == NULL )
      return new TreeNode(x,null,null);
  if (x == T.Element)
      return T;
  if (x < T.Element)
     T.Left = insert(x, T.Left);
  else T.Right = insert(x, T.Right);
  return T;
}
```



What is the running time ?

Delete a Key



How do you delete:

17 ?

9 ?

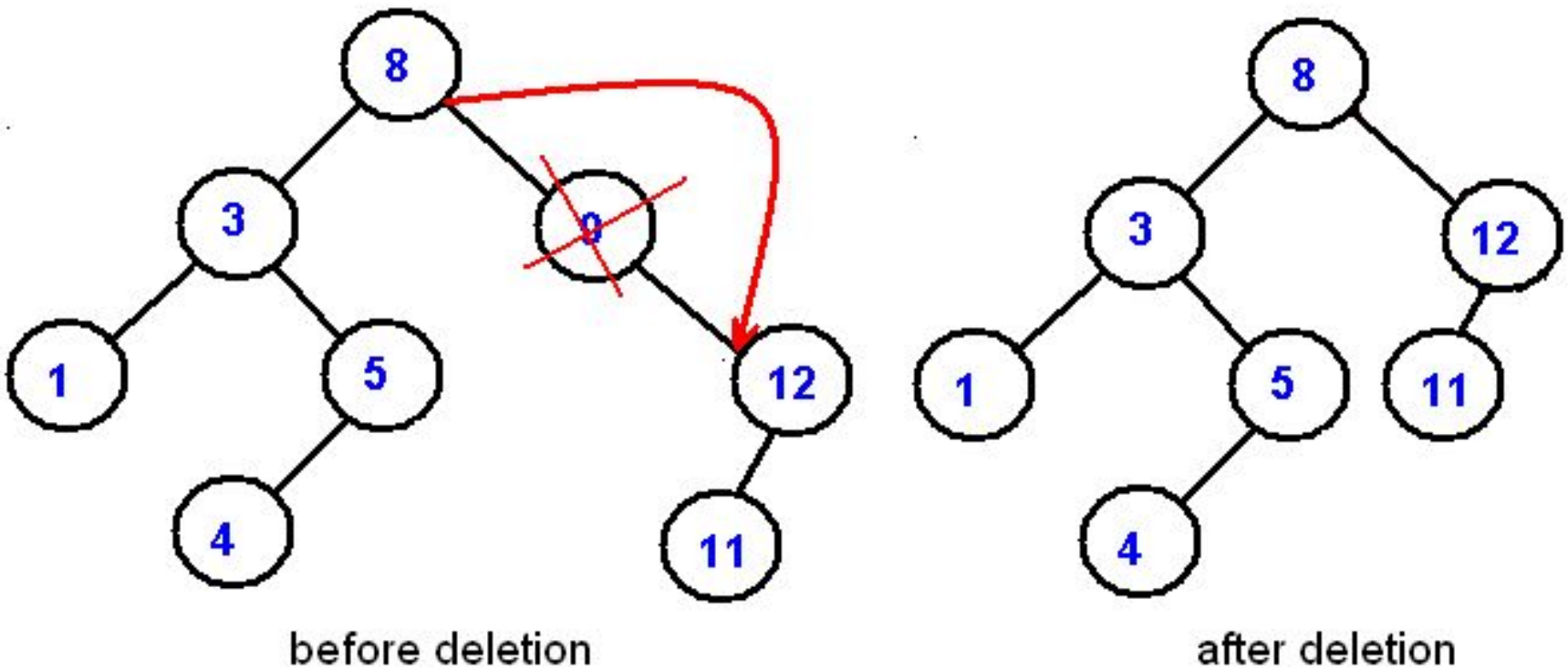
20 ?????

Deletion

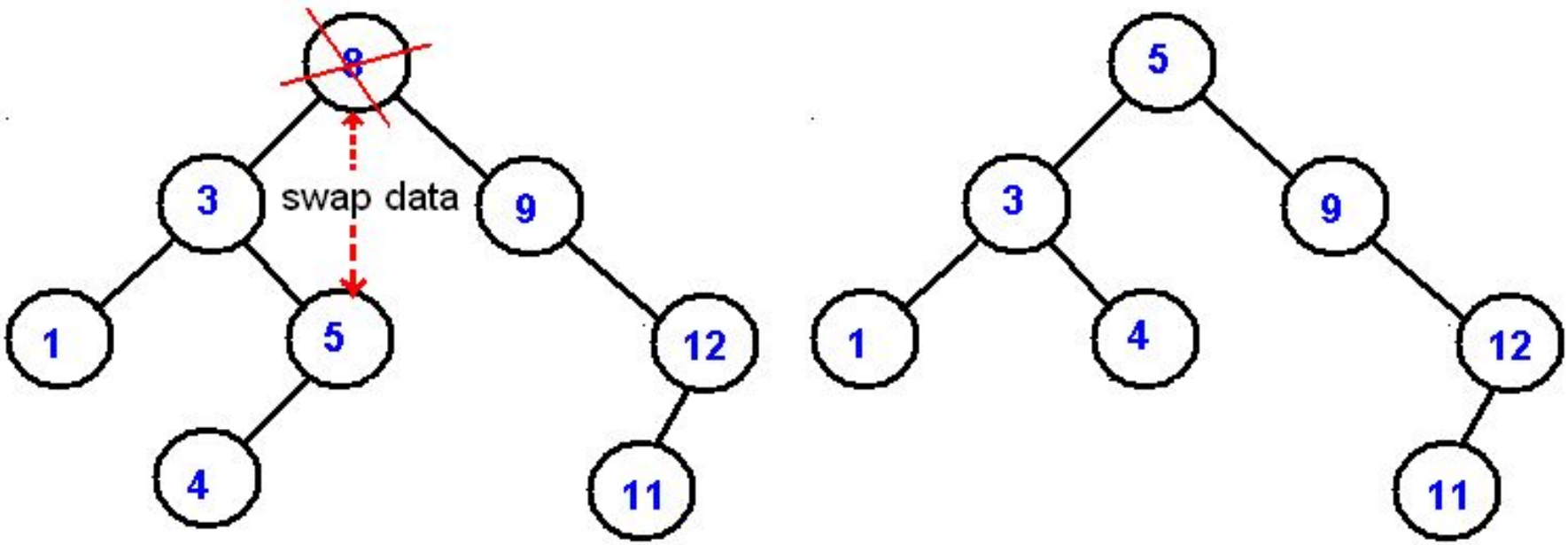
Deletion is somewhat more tricky than insertion. There are several cases to consider. A node to be deleted (let us call it as toDelete)

- is not in a tree;
- is a leaf;
- has only one child;
- has two children.

If toDelete is not in the tree, there is nothing to delete. If toDelete node has only one child the procedure of deletion is identical to deleting a node from a linked list - we just bypass that node being deleted



Deletion of an internal node with two children is less straightforward. If we delete such a node, we split a tree into two subtrees and therefore, some children of the internal node won't be accessible after deletion. In the picture below we delete 8:



Deletion starategy is the following: replace the node being deleted with the largest node in the left subtree and then delete that largest node. By symmetry, the node being deleted can be swapped with the smallest node is the right subtree.

For visualization of BST , click on the following link.

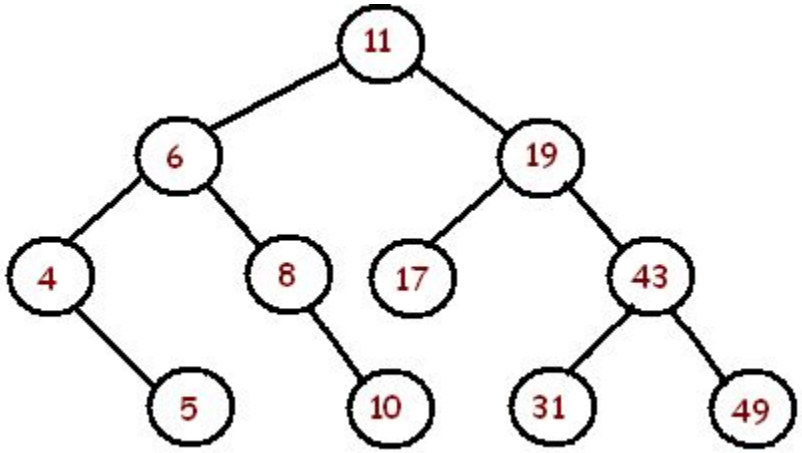
https://www.cs.usfca.edu/~galles/visualization/BST.html

Exercise. Given a sequence of numbers:

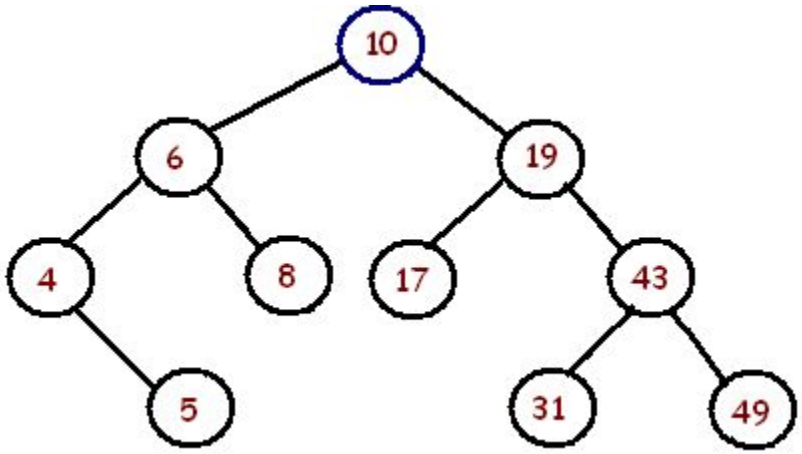
11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31

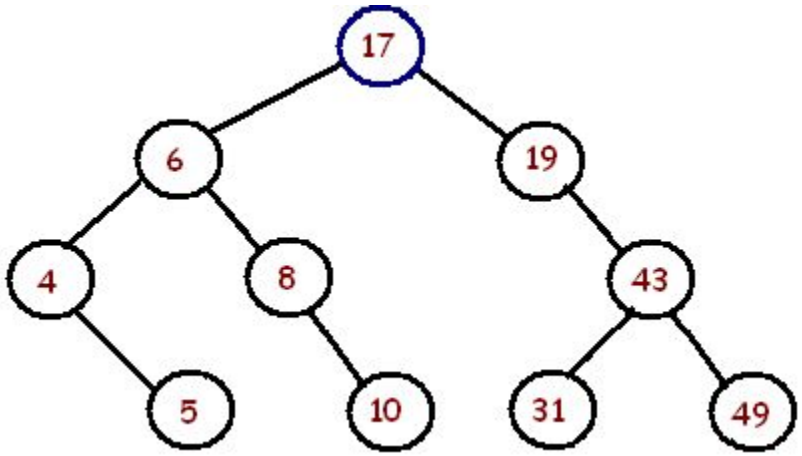
A) Draw a binary search tree by inserting the above numbers from left to right and (B) then show the two trees that can be the result after the removal of 11.

A

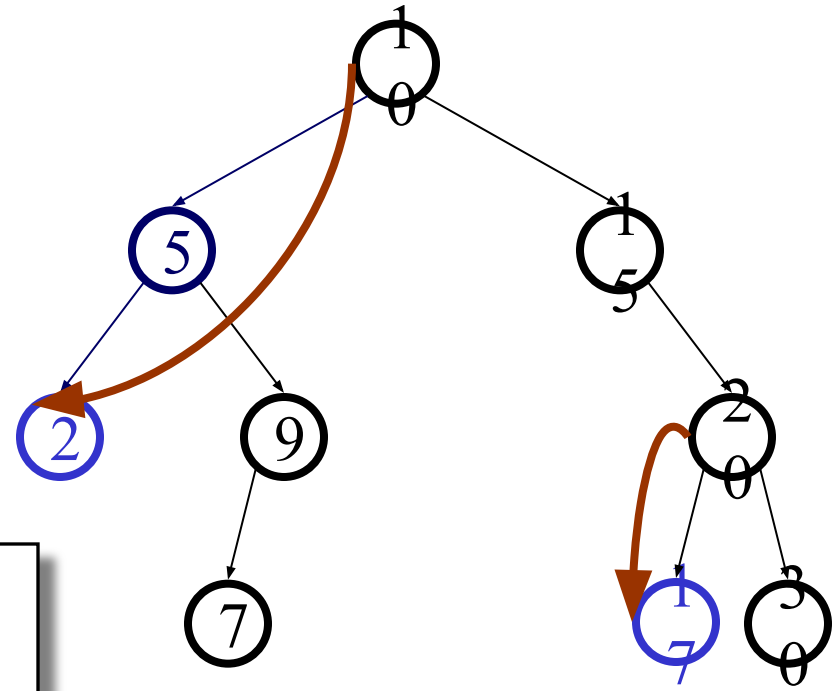


B





FindMin




```
TreeNode min(Node T) {
  if (T.Left == NULL)
    return T;
  else
    return min(T.Left); }
```

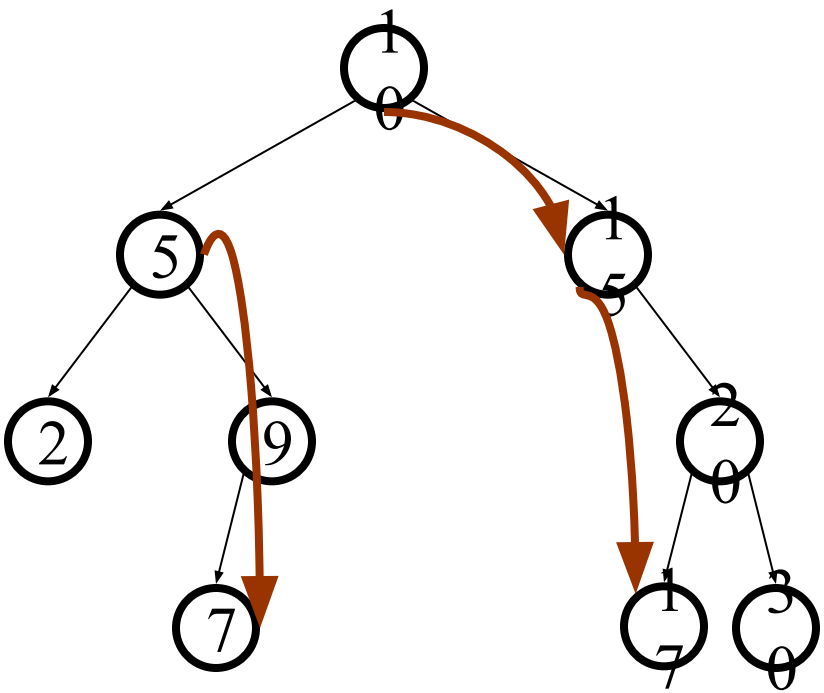
How many children can the min of a node have?

Successor

Find the next larger node in this node's subtree.

- *When it exists, it is the next largest node in entire tree*

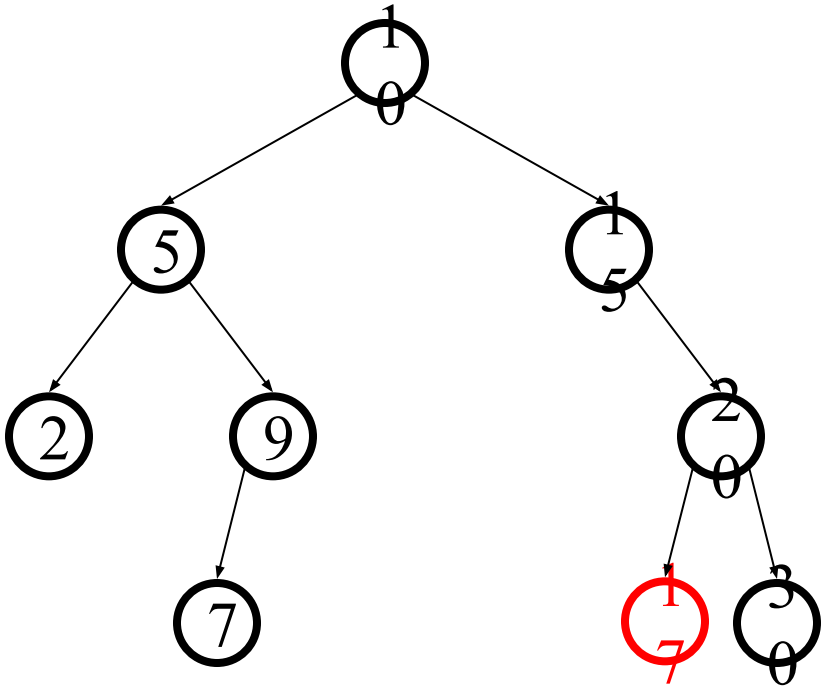
```
TreeNode succ(TreeNode T) {
  if (T.right == NULL)
    return NULL;
  else
    return min(T.right);
}
```



How many children can the successor of a node have?

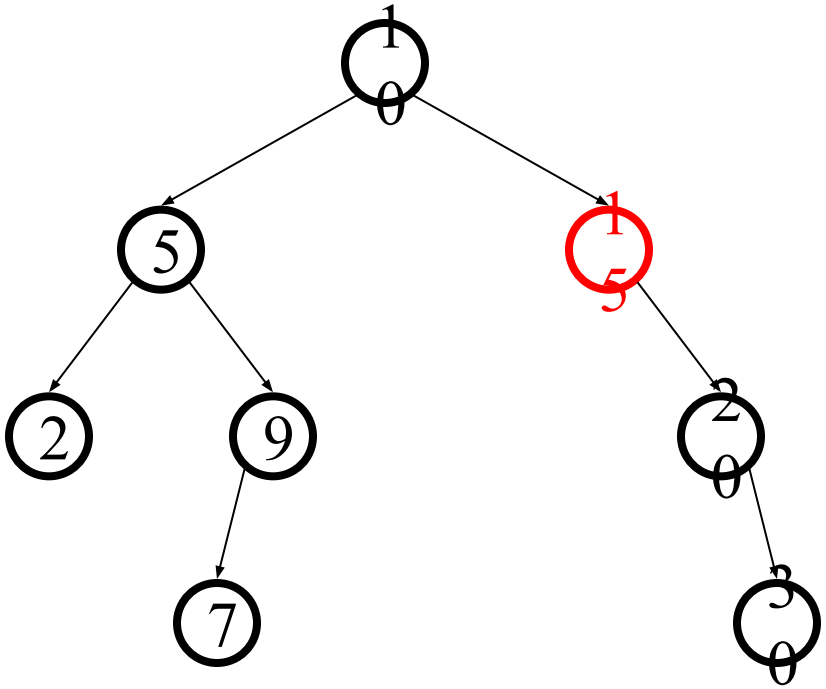
Deletion - Leaf Case

Delete(17)



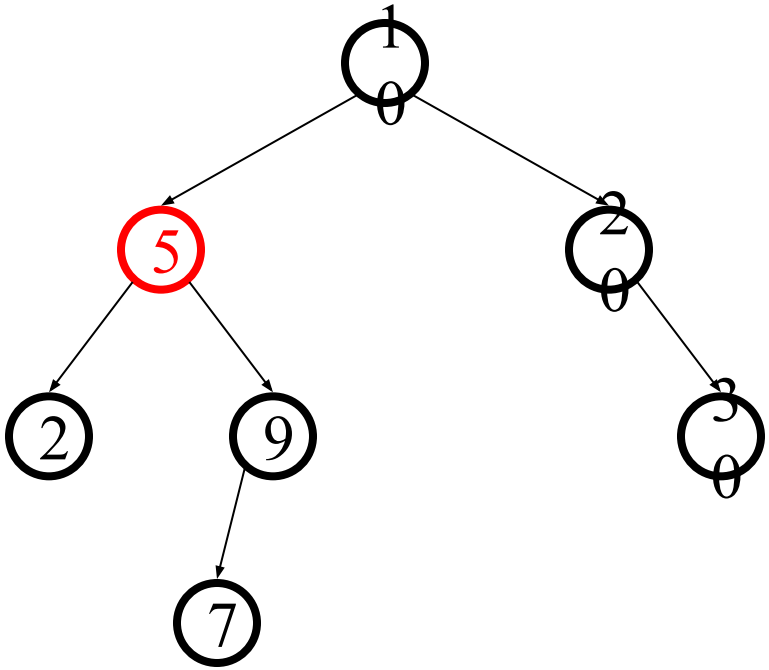
Deletion - One Child Case

Delete(15)



Deletion - Two Children Case

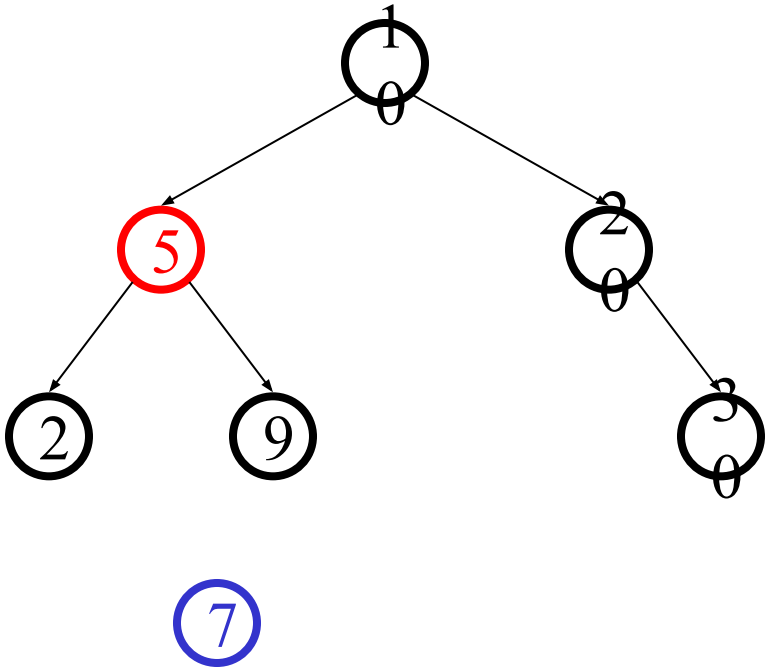
Delete(5)



replace node with value guaranteed to be between the left and right subtrees: the successor

Deletion - Two Children Case

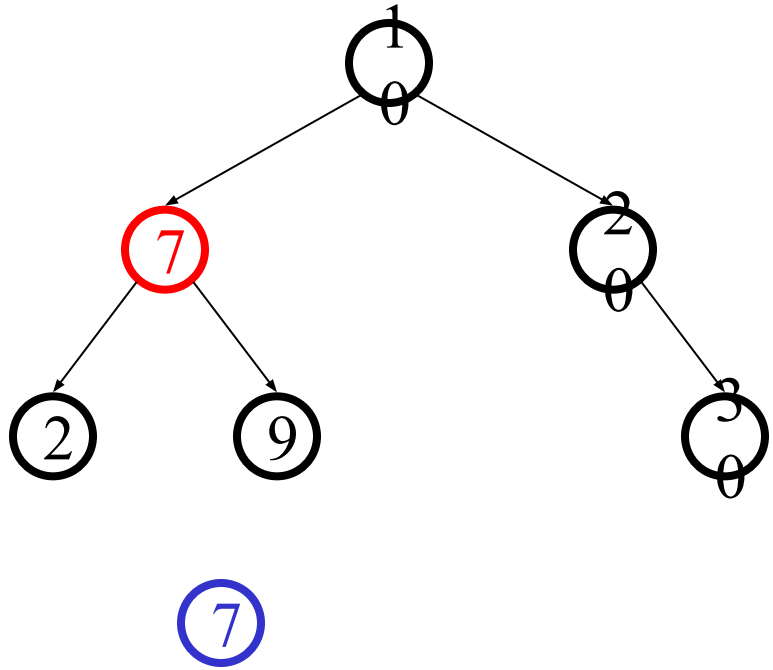
Delete(5)



always easy to delete the successor – always has either 0 or 1 children!

Deletion - Two Child Case

Delete(5)



Finally copy data value from deleted successor into original node

What is the cost of a delete operation ?

Can we use the *predecessor* instead of successor ?

Cost of the Operations

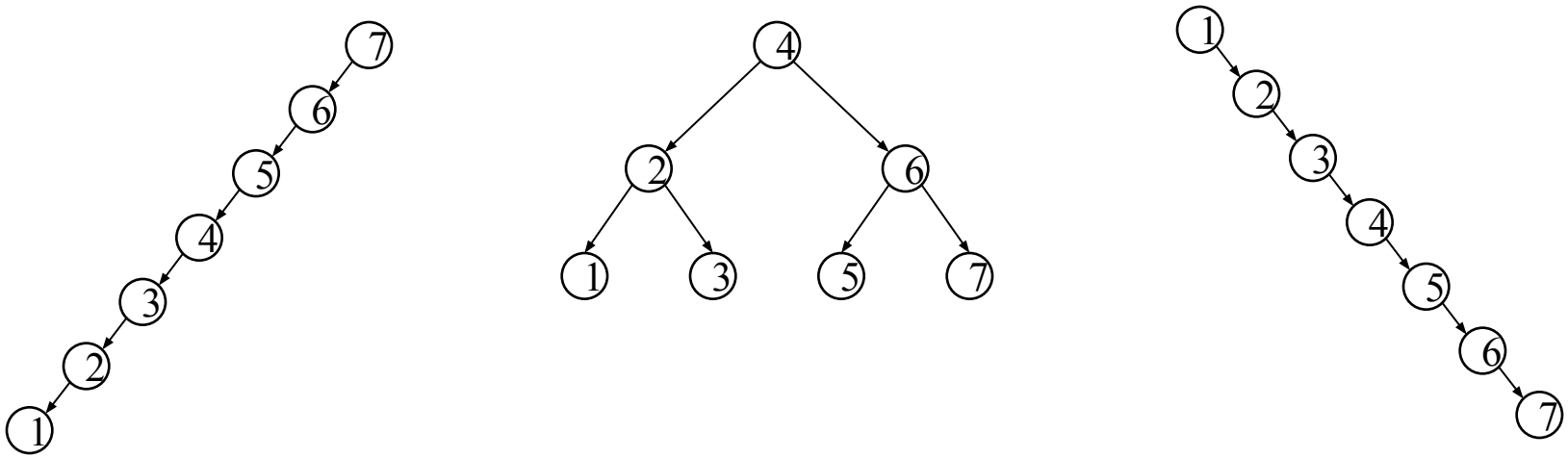
- find, insert, delete : $\boxed{\text{time} = O(\text{height}(T))}$

- Need to compute height(T)

- For a tree T with n nodes:
  - height(T) $\leq$ n
  - height(T) $\geq$ log(n) (why ?)

Height of the Binary Search Tree

- Height depends critically on the order in which we insert the data:
  - E.g. 1,2,3,4,5,6,7 or 7,6,5,4,3,2,1, or 4,2,6,1,3,5,7



Traversing the tree

There are mainly three types of tree traversals.

Pre-order traversal

- Process data of root node
- First, traverse left subtree completely
- Then, traverse right subtree

Post-order traversal

- Process data of left subtree
- First, traverse right subtree
- Then, traverse root node

In-order traversal

- First process left subtree (before processing root node)
- Then, process current root node
- Process right subtree

Heaps

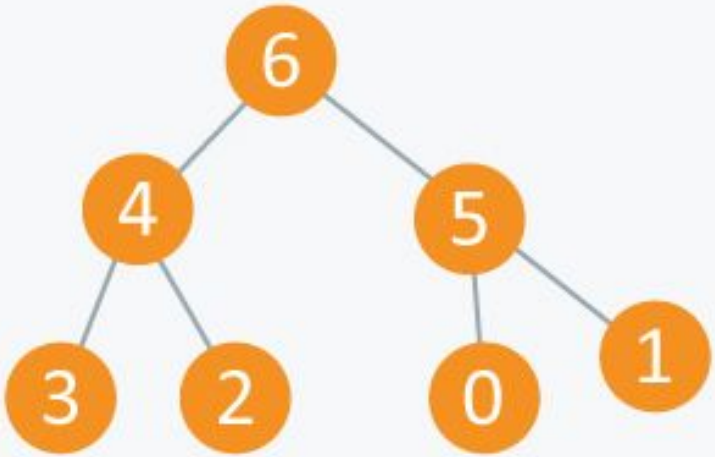
A heap is a tree-based data structure in which all the nodes of the tree are in a specific order.

For example, if **X** is the parent node of **Y** , then the value of **X** follows a specific order with respect to the value of **Y** and the same order will be followed across the tree.

The maximum number of children of a node in a heap depends on the type of heap. However, in the more commonly-used heap type, there are at most **2** children of a node and it's known as a Binary heap.

In binary heap, if the heap is a complete binary tree with **n** nodes, then it has smallest possible height which is $\log_2(n)$

.



In the diagram above, you can observe a particular sequence, i.e each node has greater value than any of its children.